NETWORK STUDY · WEEK 2

흐름 제어와 혼잡 제어

TCP는 한 번에 얼마나 보낼지
무엇을 보고 정할까

유진영 · 2026년 10월 27일 · 《혼자 공부하는 네트워크》 04-3

TCP의 약속: 빠짐없이, 순서대로

1

번호를 붙여 보낸다

데이터를 잘게 나눈 조각(세그먼트)마다 순서 번호를 붙인다. 받는 쪽은 번호를 보고 순서를 맞춘다.

2

받으면 답장한다 (ACK)

"여기까지 잘 받았어요, 다음은 N번 주세요"라는 답장. 오늘 나오는 두 제어 모두 이 답장을 이용한다.

3

답장이 없으면 다시 보낸다

일정 시간 안에 답장이 안 오면 중간에 사라졌다고 보고 그 조각을 다시 보낸다(재전송).

UDP는 이 셋이 없다. 그래서 빠르지만, 사라진 데이터는 앱이 알아서 해야 한다.

하나씩 답장을 기다리면 너무 느리다

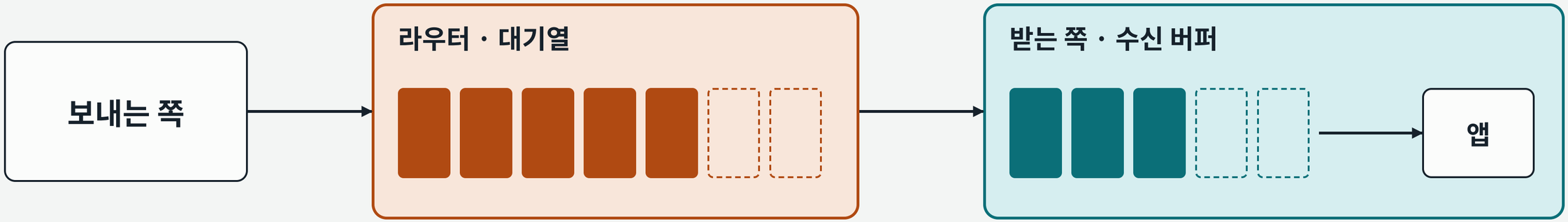


왕복 시간이 50ms이고 조각 하나가 약 1.5KB라면, 하나씩 기다릴 때 1초에 20번 왕복해서 **약 30KB/s**. 기가 인터넷이어도 이 속도에 묶인다.

그래서 TCP는 답장 전에 여러 개를 보낸다. 오늘의 질문은

그럼 몇 개까지 연달아 보내도 될까?

많이 보내면 넘치는 곳이 두 군데 있다



넘치면 라우터가 버린다

여러 사람이 같이 쓰는 길목. 몰리면 줄을 세웠다가, 줄이 꽉 차면 그냥 버린다. → 혼잡 제어

넘치면 더 받을 수 없다

도착했지만 앱이 아직 안 읽어 간 데이터를 운영체제가 잠시 담아 두는 곳. → 흐름 제어

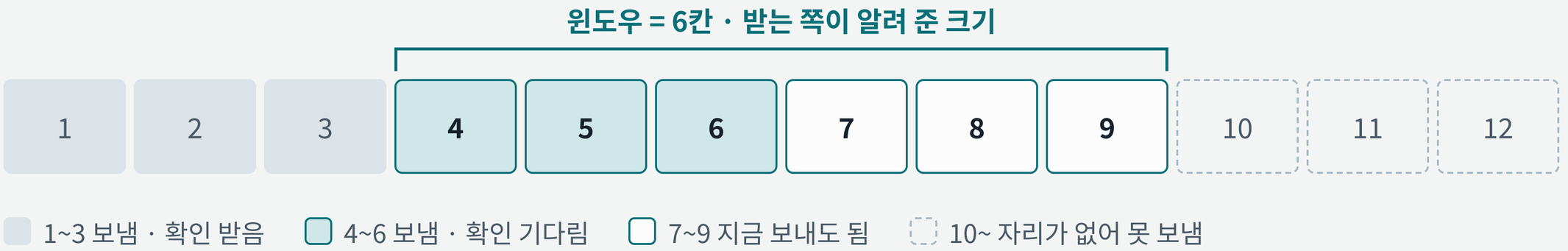
둘 다 덜 보내게 하지만, 지키는 대상이 다르다

	흐름 제어	혼잡 제어
지키는 것	받는 쪽 컴퓨터	중간의 네트워크 (라우터들)
막으려는 일	받는 쪽 버퍼가 넘쳐 버려지는 것	라우터 대기열이 넘쳐 패킷이 버려지는 것
누가 알려 주나	받는 쪽이 직접 알려 준다	아무도 안 알려 준다. 보내는 쪽이 짐작
조절하는 값	수신 윈도우 (rwnd)	혼잡 윈도우 (cwnd)

한 줄로: 흐름 제어는 상대를 배려하고, 혼잡 제어는 길을 배려한다

흐름 제어: 받는 쪽이 남은 자리를 알려 준다

받는 쪽은 ACK를 보낼 때마다 TCP 헤더의 윈도우 칸에 버퍼에 남은 자리를 적는다. 보내는 쪽은 아직 확인받지 못한 데이터를 그 크기 안에서만 둔다.



4번의 ACK가 오면 창 전체가 한 칸 오른쪽으로 밀리고 10번을 보낼 수 있게 된다. 그래서 슬라이딩 윈도우.

자리가 0이 되면 보내는 쪽은 멈춘다

1. 받는 쪽 앱이 데이터를 늦게 읽어 간다
2. 수신 버퍼가 가득 차서 윈도우 0을 알린다
3. 보내는 쪽은 멈추고, 가끔 작은 패킷으로 자리가 났는지 찔러 본다

와이어샤크에서는 TCP ZeroWindow로 표시된다

앱에서는 백프레셔라고 부른다

- Node 스트림의 write()가 false를 돌려주면 drain 이벤트까지 기다린다
- 브라우저 WebSocket의 bufferedAmount가 계속 커지면 전송이 못 따라가는 것
- 느린 클라이언트에 SSE를 계속 밀어 넣으면 서버 메모리에 쌓인다

혼잡 제어: 길 사정은 아무도 안 알려 주니 짐작한다

답장 없이 연달아 보낼 수 있는 양

= 받는 쪽 빈자리와 내 짐작 중 더 작은 쪽

`min( 수신 윈도우 rwnd, 혼잡 윈도우 cwnd )`

짐작의 단서: 답장이 안 온다

라우터 대기열이 넘쳐 데이터가 버려지면, 그 데이터의 답장(ACK)이 오지 않는다. 보내는 쪽은 이걸 보고 "길이 막혔구나" 하고 보내는 양을 줄인다.

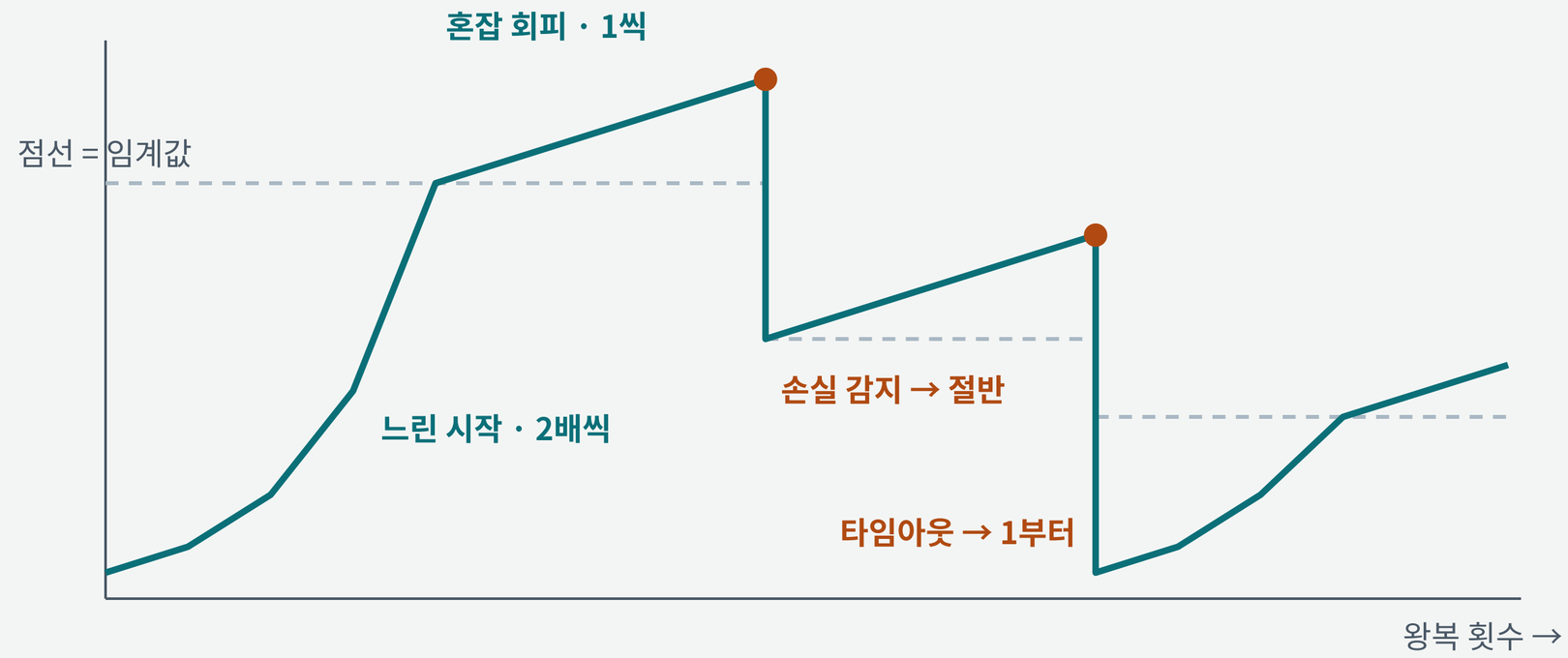
예: 1~5번을 보냈는데 2번만 사라지면, 받는 쪽은 3·4·5번을 받을 때마다 "2번 주세요"를 보낸다. 같은 답장이 세 번 오면 2번이 사라졌다고 본다.

더 알아보기 (오늘은 이름만)

지연 증가: 답장이 점점 늦게 오는 것도 단서로 쓴다 (BBR)

ECN: 라우터가 버리기 전에 "막힘" 표시를 붙여 준다

늘릴 때는 천천히, 문제가 생기면 크게 줄인다



느린 시작

작게 시작해 왕복마다 두 배로 늘린다. 이름과 달리 빠르게 커진다.

혼잡 회피

임계값을 넘으면 왕복마다 조금씩만 늘린다.

손실이 나면

같은 ACK 3번이면 절반으로, 타임아웃이면 처음부터 다시.

그림의 값은 이해를 위한 예시 (TCP Reno 방식). 리눅스 기본인 CUBIC은 늘어나는 곡선 모양이 다르지만 큰 흐름은 같다.

웹 개발에서 만나는 지점

혼잡 제어

새 연결은 느리게 시작한다

연결마다 혼잡 윈도우가 작게 시작한다. 연결을 다시 쓰는 keep-alive와 HTTP/2가 빠른 이유 중 하나다.

혼잡 제어

첫 왕복에 실리는 양이 정해져 있다

리눅스 서버는 처음에 세그먼트 10개, 약 14KB를 보낸다. 첫 화면에 필요한 HTML을 작게 두면 첫 왕복에 끝난다.

흐름 제어

느린 상대에게 계속 밀어 넣지 않는다

SSE·WebSocket 서버는 클라이언트별 보낼 버퍼에 한도를 두고, 넘으면 끊거나 오래된 메시지를 버린다.

keep-alive란

요청 하나가 끝나도 연결을 끊지 않고 다음 요청에 다시 쓰는 것. 전화를 끊지 않고 "그리고 하나 더요" 하고 이어 말하는 것과 같다.

HTTP/2란

연결 하나로 여러 요청을 동시에 섞어 주고받는 HTTP 버전. 전화 한 통으로 여러 주문을 한꺼번에 넣는 것과 같다.

정리

1. 흐름 제어는 받는 쪽을, 혼잡 제어는 네트워크를 지킨다.
2. 흐름 제어는 받는 쪽이 윈도우로 직접 알려 주고, 혼잡 제어는 보내는 쪽이 손실과 지연으로 짐작한다.
3. 실제로 보내는 양은 두 윈도우 중 작은 쪽이다.
4. 새 연결은 작게 시작하니 연결은 다시 쓰고, 느린 상대에게는 백프레셔를 건다.

우리 서비스에서는

[우리 API 서버와 정적 파일이 keep-alive·HTTP/2로 나가는지 DevTools Protocol 열로 확인한 결과]